Django Caching, Cookies & Sessions

1. Django Caching

Performance Optimization

Caching হলো frequently accessed data সাময়িকভাবে store করে রাখা, যাতে প্রতিবার database থেকে data fetch করতে না হয়।

কেন Caching ব্যবহার করব?

ধরো, তোমার Django application-এ একটি API আছে:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Product


@api_view(["GET"])
def product_list(request):
    products = Product.objects.all()

    data = [
        {
            "id": product.id,
            "name": product.name,
            "price": product.price
        }
        for product in products
    ]

    return Response(data)
```

প্রতিবার API call হলে database query execute হবে।

Caching ব্যবহার করলে প্রথমবার database থেকে data নিয়ে cache-এ রাখা হবে। পরবর্তী request-গুলো cache থেকে data পাবে।

Without Cache

1. User Request
2. Django View
3. Database Query (Every Request)

With Cache

1. User Request
2. Django View
3. Cache Check
4. Cache Hit → Return Cached Data
5. Cache Miss → Database Query

1.1 Django Cache Backends

Django বিভিন্ন ধরনের cache backend support করে।

Backend	Description

LocMemCache	Local memory caching
FileBasedCache	File system-এ cache store
DatabaseCache	Database table-এ cache store
Memcached	Distributed memory cache
Redis	External cache backend (compatible backend দিয়ে)

Recommended Cache Backend

Production Recommendation: Redis

Redis production applications-এ খুব জনপ্রিয়। কারণ এটি fast in-memory data store এবং multiple application instances-এর মধ্যে shared cache হিসেবে কাজ করতে পারে।

Step 1: Install Redis Client

```
pip install django-redis
```

Redis server-ও চালু থাকতে হবে।

Step 2: settings.py Configuration

```
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
        "OPTIONS": {
            "CLIENT_CLASS": "django_redis.client.DefaultClient",
        },
        "TIMEOUT": 300,
    }
}
```

এখানে:

- **BACKEND:** Cache backend।
- **LOCATION:** Redis server address।
- **TIMEOUT:** Cache কতক্ষণ থাকবে (seconds)।
- **300:** 5 minutes।

1.2 Django Caching Strategies

Django-তে চারটি গুরুত্বপূর্ণ caching strategy আছে।

1. Per-site Cache

সম্পূর্ণ website-এর response cache করে।

settings.py

```
MIDDLEWARE = [
    "django.middleware.cache.UpdateCacheMiddleware",
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.cache.FetchFromCacheMiddleware",
]
```

সাধারণত UpdateCacheMiddleware middleware list-এর প্রথমে এবং FetchFromCacheMiddleware শেষে রাখতে হয়।

```
CACHE_MIDDLEWARE_SECONDS = 600
```

2. Per-view Cache

নির্দিষ্ট একটি view-এর response cache করে।

```
from django.views.decorators.cache import cache_page


@cache_page(60 * 5)
def product_list(request):
    products = Product.objects.all()

    return render(
        request,
        "products.html",
        {"products": products}
    )
```

এখানে response 5 মিনিট cache থাকবে।

DRF API-তেও cache ব্যবহার করা যায়।

```
from django.views.decorators.cache import cache_page
from django.utils.decorators import method_decorator
from rest_framework.views import APIView


@method_decorator(cache_page(60 * 5), name="dispatch")
```

```
class ProductListAPIView(APIView):

    def get(self, request):
        products = Product.objects.all()

        # Return serialized response
```

3. Template Fragment Cache

Template-এর নির্দিষ্ট অংশ cache করে।

```
{% load cache %}

<h1>Products</h1>

{% cache 300 featured_products %}
    {% for product in products %}
        <p>{{ product.name }}</p>
    {% endfor %}
{% endcache %}
```

4. Low-level Cache API

নিজের প্রয়োজন অনুযায়ী নির্দিষ্ট data cache করা যায়।

```
from django.core.cache import cache

# Store data
cache.set("username", "Faruk", timeout=300)

# Retrieve data
username = cache.get("username")

# Delete data
cache.delete("username")

# Check existing key
if cache.get("username") is not None:
    print("Cache exists")
```

Important Cache Methods

Method	কাজ
cache.set()	Data store
cache.get()	Data retrieve
cache.add()	Key না থাকলে store

cache.delete()	Delete one key
cache.clear()	Clear entire cache
cache.get_or_set()	Get or create cache
cache.incr()	Increment value
cache.decr()	Decrement value

1.3 Real-world Cache Example

একটি Product API cache করা:

```
from django.core.cache import cache
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Product


@api_view(["GET"])
def product_list(request):

    cache_key = "product_list"

    products = cache.get(cache_key)

    if products is None:
        products = list(
            Product.objects.values(
                "id",
                "name",
                "price"
            )
        )

        cache.set(cache_key, products, timeout=300)

    return Response(products)
```

এখানে:

- প্রথম request → Database query।
- পরবর্তী request → Cache থেকে response।
- 5 মিনিট পরে → Cache expire হবে।

Important: Product create, update বা delete হলে সংশ্লিষ্ট cache invalidate করতে হবে।

---

2. Django Cookies

Client-side Storage

Cookie হলো ছোট data যা browser-এর মধ্যে store হয় এবং নির্দিষ্ট request-এর সঙ্গে server-এ পাঠানো হয়।

উদাহরণ

- User preferences
- Language selection
- Theme selection
- Remember-me functionality
- Tracking identifiers

2.1 Set Cookie

Django view থেকে cookie তৈরি করা যায়।

```
from django.http import HttpResponse


def set_cookie(request):

    response = HttpResponse("Cookie Created")

    response.set_cookie(
        "username",
        "Faruk",
        max_age=3600
    )

    return response
```

`max_age=3600` অর্থ cookie 1 ঘণ্টা থাকবে।

2.2 Get Cookie

```
from django.http import HttpResponse


def get_cookie(request):

    username = request.COOKIES.get("username")

    return HttpResponse(f"Username: {username}")
```

2.3 Delete Cookie

```
def delete_cookie(request):

    response = HttpResponse("Cookie Deleted")

    response.delete_cookie("username")

    return response
```

2.4 Important Cookie Attributes

```
response.set_cookie(
    "user_token",
    "example_token",
    max_age=3600,
    httponly=True,
    secure=True,
    samesite="Lax"
)
```

Attribute	কাজ
max_age	Cookie lifetime
expires	Expiration date
httponly	JavaScript access বন্ধ
secure	HTTPS ছাড়া পাঠাবে না
samesite	Cross-site request control
path	কোন URL path-এ cookie পাঠাবে
domain	কোন domain-এ cookie পাঠাবে

Security Tip: Sensitive information যেমন password বা plain authentication token সাধারণ cookie-তে রাখবে না। HTTPS, HttpOnly, Secure এবং উপযুক্ত SameSite policy ব্যবহার করবে।

---

3. Django Sessions

Server-side User State

Session হলো server-side-এ user-specific data store করার একটি system।

উদাহরণ

- User login state
- Shopping cart
- Temporary user information
- Multi-step form progress

- User-specific preferences

Session কীভাবে কাজ করে?

1. User website-এ request পাঠায়।
2. Django একটি session তৈরি করে।
3. Session data server-side storage-এ save হয়।
4. Browser-এ session ID cookie পাঠানো হয়।
5. পরবর্তী request-এ browser session ID পাঠায়।
6. Django session ID দিয়ে user-এর data retrieve করে।

3.1 Session Configuration

Django-তে default session configuration সাধারণত আগে থেকেই থাকে।

settings.py

```
INSTALLED_APPS = [
    "django.contrib.sessions",
]

MIDDLEWARE = [
    "django.contrib.sessions.middleware.SessionMiddleware",
]
```

Migration চালাও:

```
python manage.py migrate
```

Default session backend হলো database-backed session।

3.2 Set Session

```
from django.http import HttpResponse


def set_session(request):

    request.session["username"] = "Faruk"
    request.session["user_id"] = 101

    return HttpResponse("Session Created")
```

3.3 Get Session

```
def get_session(request):

    username = request.session.get("username")

    return HttpResponse(f"Welcome {username}")
```

3.4 Delete Session Data

```
def delete_session(request):

    request.session.pop("username", None)

    return HttpResponse("Session Data Deleted")
```

3.5 Clear All Session Data

```
  def clear_session(request):

      request.session.flush()

      return HttpResponse("Session Cleared")
```

`flush()` session data clear করার পাশাপাশি current session key-ও সরিয়ে দেয়।

3.6 Session Expiry

```
  def create_session(request):

      request.session["username"] = "Faruk"

      # Session expires after 30 minutes
      request.session.set_expiry(1800)

      return HttpResponse("Session Created")
```

আরও কিছু expiry option:

Browser বন্ধ হলে session expire হবে।

```
  request.session.set_expiry(0)
```

Global session expiry setting ব্যবহার করবে।

```
  request.session.set_expiry(None)
```

---

4. Cookies vs Sessions vs Caching

এটি Interview-এর জন্য অত্যন্ত গুরুত্বপূর্ণ।

Feature	Cookies	Sessions	Caching
Storage	Browser	Server-side (usual setup)	Cache backend
Purpose	Client data	User state	Performance
Lifetime	Configurable	Configurable	TTL
Security	Client-controlled	Session ID protection প্রয়োজন	Backend security প্রয়োজন
Example	Theme	Login state	Product data

সহজভাবে মনে রাখবে

- **Cookies** → Browser-এর data।
- **Sessions** → User-specific server-side state।
- **Caching** → দ্রুত data access করার ব্যবস্থা।

একটি গুরুত্বপূর্ণ বিষয়: Django-র default session backend database-based হলেও cache-based session backend-ও আছে। আবার cookie-based session backend-ও আছে।

---

5. Real-world Django Project Example

ধরো তুমি একটি E-commerce application তৈরি করছ।

Feature	কোনটি ব্যবহার করবে?
Product list দ্রুত load	Redis Cache
Product details cache	Per-view / Low-level Cache
User login state	Django Session
Shopping cart	Session অথবা Database
Dark/Light theme	Cookie
Language preference	Cookie
Temporary OTP verification state	Session with appropriate security

Recommended Architecture

```
Browser / Frontend
        |
        |
Cookies + Session ID
        |
        |
```

```
Django Backend / DRF
        |
        |
Views, APIs, Authentication
        |
        |
   +----+----+
   |         |
 Redis   PostgreSQL
   |         |
 Cache   Persistent Data
```

---

6. Interview Questions & Answers

Q1. What is caching in Django?

Caching stores frequently accessed data temporarily to reduce database queries and improve application performance.

Q2. What is Redis?

Redis is an in-memory data store commonly used for caching, session storage, and other fast data operations.

Q3. What is the difference between cache and database?

Database stores persistent application data, whereas cache stores temporary data to improve performance.

Q4. What is a cookie?

A cookie is a small piece of data stored in the user's browser.

Q5. What is a session?

A session stores user-specific state, usually on the server, identified through a session ID.

Q6. Can sessions work without cookies?

Django's standard session middleware uses cookies to transmit the session key. A custom session mechanism could use another transport.

Q7. What is HttpOnly?

It prevents JavaScript from reading the cookie through browser APIs.

Q8. What is the difference between cache.get() and cache.get_or_set()?

- `cache.get()` retrieves an existing value.
- `cache.get_or_set()` retrieves an existing value or creates and stores one if missing.

Q9. What is cache invalidation?

Removing or updating cached data when the original data changes.

Q10. Where should authentication information be stored?

Prefer Django's established authentication/session mechanisms or an appropriately designed token authentication system, rather than storing passwords or sensitive data in ordinary cookies.

---

7. Practical Learning Roadmap

তোমার Django Backend Development শেখার জন্য এই sequence follow করো।

Your Practice Checklist

- **1. Understand Django Cache Backends**
  - LocMemCache

    - Redis
    - DatabaseCache
- **2. Practice Low-level Cache API**
    - set()
    - get()
    - delete()
    - get_or_set()
- **3. Implement Redis in Django REST Framework**
    - Cache Product API responses
- **4. Learn Cookies**
    - Set
    - Get
    - Delete
    - Cookie security
- **5. Learn Sessions**
    - Create
    - Read
    - Update
    - Delete
    - Expiry
- **6. Build an E-commerce Cart**
    - Session-based shopping cart
- **7. Implement Cache Invalidation**
    - Invalidate cache after Product updates

Final Recommendation

আমার recommendation: প্রথমে Django Low-level Cache API ভালোভাবে শিখবে, এরপর Redis integration করবে। তারপর Cookies এবং Sessions দিয়ে একটি complete authentication ও shopping cart workflow তৈরি করবে।
এভাবে শিখলে শুধু interview নয়, real-world Django Backend Developer হিসেবেও কাজে লাগবে।